

Java 面试前速记

2026.09.22 14:30 | 手机阅读版

面试：2026 年 9 月 22 日 14:30。先看第一节，剩余内容按需扫一遍，不再背新知识。本文按最近口述整理；“通用答法”不是在宣称项目已经全部实现。

5分钟路线：第1节总览 → 第5节幂等 → 第7节排障 → 第9节反问。 有余力再扫项目和Java提纲，不要求全文背完。

1. 只剩一分钟，就看这里

- **先结论 → 原因 → 一个项目例子 → 必要边界。** 听完停两秒，不急着堆术语。
- **我的定位：7年Java | 中软雇主、海思客户 | 用户中心负责人 + 亲自攻关 | 制造域DDD重构 | AI业务集成。**
- **同一次请求同一个号，新业务用新号；数据库唯一约束兜底。先查不原子，锁不等于幂等。**
- **超时≠失败，查不到≠没执行；查询结果不能代替下游幂等。**
- **本地事务不管远程已提交的事务；Outbox保证业务与任务同事务落库，不保证外部只执行一次。**
- **工单上传：定时按供应商打包。公共Outbox + CDC：另一套组件。**
- **花聊：AI建草稿 → 既有微前端展示 → 用户确认 → 页面携认证信息直调Java → 后端校验执行。**
- **QPS/TPS没核实就不换名、不主动报十倍；活跃连接高不等于连接池已经耗尽。**
- **不确定时说：“这部分我能确认的是.....具体配置/统计口径需要核对，不能给您一个不准确的数字。”**

2. 开场与求职原因

自我介绍：记主线，不背逐字稿

我叫宋凯旋，有7年Java开发经验，目前就职于中软国际，主要承担华为海思供应链系统的核心开发。

我曾担任用户中心项目负责人，负责需求分析、方案设计、任务安排和交付推进，也亲自承担JALOR权限框架适配、租户权限隔离等核心改造。项目高质量提前交付，获得了客户方领导的公开表扬。

另外，我参与供应链基于DDD的去Oracle重构，负责制造领域的工单、工单上传、加工计划相关设计开发和存储过程迁移。今年还参与了供应链AI助手建设，负责SSE、会话相关工作和已有服务能力的接入维护功能。希望继续承担Java核心开发，发挥复杂业务分析和技术攻关的经验。

为什么换工作

在当前项目近五年，我承担的职责和积累的经验都有明显增长。目前重构也接近尾声，希望寻找能够长期投入的项目，同时让职责、薪酬和工作安排更匹配。

追问外包：“我也关注过甲方岗位，但会结合具体项目、工作内容和整体待遇判断，不只看用工形式。”

不必提金葵花卡，不贬低原公司，不把招行项目说成招行编制；不预先承诺无限加班。对合理上线和紧急问题愿意配合，对长期工作安排应了解清楚。

3. 用户中心：负责人 + 技术攻关

负责人：需求对齐、方案设计、任务拆分、进度/风险、QC和指导、测试上线协作。成果是提前高质量交付与公开表扬，不只说“做开发”。

框架适配：阅读JALOR源码和表结构 → 补充关系表 → 组合原能力和扩展方法。接口权限点与页面/功能关联、数据范围上报，简化角色维护和申请。说“基于原框架扩展”，不说“从零重写完整框架”。

租户隔离：原角色授权与数据维度平行 → 补用户授权与租户关系 → 申请时先选租户 → 请求头传目标租户，网关透传上下文 → 根据可信用户身份查询授权，再按目标租户过滤 → 服务端校验。

- 缓存键是用户ID，缓存值是**跨租户全部授权**，取出后按目标租户过滤；不是缓存了某一个租户结果却忘加租户键。
- 验证：菜单/按钮变化、权限返回值、Postman直接请求被拒绝。**页面隐藏不是权限安全边界。**
- 网关部分实际用过同包同名类替换；不能把所有改造都讲成继承。类加载优先级依赖部署，升级要验证，不猜配置。

QC例子：多人审批误用了仅初始成立的状态，第一人审批后其余人不能操作 → 提修改意见 → 改后由提意见者复审 + 页面功能演示。不要补造未做过的自动化测试。

权限查询优化

用户反馈页面转圈 → F12发现权限接口慢 → 分段日志定位数据库访问段 → Redis减少重复查库 + Caffeine减少热点网络访问和共享Redis压力 → 压测、上线监控和用户反馈验证。

数字口径待核对：200→2000+到底是QPS还是TPS记不清，简历被追问时坦诚说明。不要因此称全系统提速十倍；连接池“活跃数高”也不能直接升级为“耗尽”。

实际失效链路：曾做前置删Redis；业务修改 + 任务记录同事务 → 提交后CDC再次删Redis、发广播 → 实例清本地缓存。删Redis与发布广播成功后更新任务状态。

通用边界：这是最终一致；发布成功≠每个实例已消费。失败重试/扫描是可靠推进措施，真实调度频率未核实不硬报。TTL兜底，随机偏移减轻集中失效，不等于立即撤销。严格敏感操作应查权威权限、避开过期缓存，查询失败拒绝；已通过鉴权的在途操作不是自动撤销。

4. DDD、上传、迁移：别混在一起

概念：实体靠身份标识；值对象按值表达概念；聚合是保护业务不变量的一致性边界；聚合根控制修改入口。战略设计定边界/语言，战术设计落实体、聚合、服务、仓储。

拆上传：工单管单个工单的状态和流转；上传批次组织多个工单快照，有自己的规则和状态。部分工单需要上传成功才继续流转，是业务依赖，不要求合成一个聚合。拆模块不自动等于限界上下文或微服务。

接口层：接收请求、转换参数

应用层：编排用例与事务，协调读取、校验、保存

领域层：是否允许上传、业务转换、状态流转等规则

基础设施层：实现数据库访问、S3、外部接口等技术细节

本项目上传：读取工单/配置 → 领域校验 → 转换组装快照并落表 → **独立定时任务按供应商打包、生成文件上传**。避免数据库大事务覆盖S3调用。不是每新增工单立即用CDC上传。

并发流转：任务流不等于没有并发。更新带版本号，持久层发现影响0行会抛异常回滚；0行本身不会自动回滚。

迁移例子：工单生成存储过程源码分析 → PL/SQL调试查触发器/子过程 → 业务串讲 → 同输入对比结果 → Java + PG承接生产。该功能旧Oracle环境保留结果比对，关闭旧上传/消息/下游出口，差异告警。别泛化为全系统都有影子比对。

5. Outbox、超时、幂等：最后修补的重点

公共组件（项目事实）：工具方法在业务同一事务中写任务表 → CDC识别目标执行消息/接口操作 → 成功更新任务状态。

通用边界：外部成功、状态未更新就宕机，可能重复执行。可靠落库 + 重试解决任务推进；稳定业务号 + 下游幂等解决重复副作用。先查后重试仍有窗口，不能把“支持查询”当成“不需要幂等”。

超时处理：同一个请求号查询；成功复用结果，处理中继续查询；查不到也可能仍在执行，不能盲目加库存补偿。需重试时复用原号。业务确需补偿，补偿也要防重复。

订单幂等通用答法：

1. 表单生成稳定订单号/请求号；同一次提交及所有重试复用，新一笔合法订单才换号。
2. 数据库唯一约束兜底，多实例同时插入也只能创建一张；先查只是优化。
3. 处理唯一冲突、在可用事务上下文查询已有结果，不直接返回系统错误。结果查询还需校验归属。

不要按“用户 + 产品 + 供应商”认定重复；两笔合法订单也能相同。页面关闭不撤回订单，刷新列表只是体验优化。分布式锁不是必要条件，更不能替代唯一约束。

6. Java / SQL：一眼回忆

- **线程池**：核心线程 → 入队 → 队满扩到最大线程数 → 拒绝。核心2/最大4/队列3，8任务均未完成：1、2执行；3~5排队；6、7新线程执行；8被AbortPolicy抛异常拒绝。
- **拒绝策略**：Abort抛异常；CallerRuns由提交线程执行（关闭时会丢弃）；Discard丢当前任务；DiscardOldest丢队首任务再尝试。不能丢的任务先可靠落库，不靠线程池内存保命。
- **volatile**：可见性和相关有序性，不保证count++或检查后扣库存的原子性。实例同步锁this，静态同步锁Class，代码块锁指定对象。
- **ThreadLocal**：线程复用可能串上下文；提交前取上下文，工作线程显式设置，finally清理。主线程的普通ThreadLocal不自动传给池线程。
- **代理**：JDK代理实现接口，不能转成实现类；CGLIB基于继承，不能增强final方法。默认代理模式下，自调用绕过事务增强。
- **Spring默认回滚**：RuntimeException和Error；受检异常需匹配回滚规则。内部吞异常可能导致代理看不到异常。

事务传播与 SQL

假设A事务通过另一Bean代理调用B，A捕获B的 RuntimeException，且无其他异常：

REQUIRED

B结果：共用事务被标记rollback-only

A结果：不能提交，通常提交处抛UnexpectedRollbackException

REQUIRES_NEW

B结果：独立事务回滚

A结果：原事务可提交

HTTP远程调用不自动加入本地事务；把调用放最后，也挡不住远程成功后本地提交失败。

SQL：普通联合索引(tenant_id,status,create_time)，前两列等值，日期条件改为：

AND create_time >= '2026-09-21 00:00:00'

AND create_time < '2026-09-22 00:00:00'

函数妨碍该列普通范围定位，不代表整个联合索引都失效。EXPLAIN rows是估计检查行数，不是实际或返回行数。回表是二级索引再取完整行；覆盖索引可避免对应回表。

7. 排障：先证据，不猜根因

接口慢/队列增长：提交量与完成速度 → 队列、活跃线程、等待/执行耗时 → 链路分段耗时 + 连续两三份线程栈 → 再查DB慢SQL/连接等待、外部超时重试、锁竞争、CPU/GC。请求量不变但完成变慢，重点定位服务时间增长；堆变大本身不能说明是哪段慢。

MyBatis：实际SQL + 绑定参数 + 一次请求查询次数。单条5ms、循环查200次也会慢；收集ID批量查，再Map组装，关联查询需注意分页和一对多。

CPU高：`top`定位PID → `top -H -p <PID>`找热线程TID → 十进制TID转十六进制 → `jstack <PID>`找nid → 多份栈看热点是否持续，再区分计算、循环、GC。生产抓取按权限和运行环境执行。

OOM：先区分堆/元空间/直接内存等 → GC日志 → 对应证据；堆问题看dump的实例分布、保留大小和GC Roots路径。泄漏看“本该释放却被长期持有”，不看引用层级深就下结论。

8. 花聊与AI：讲自己真正负责的部分

本人：SSE流式输出、会话相关工作、供IT维护已有Java/微服务能力并接入MCP的页面；不是整个Agent框架作者。

创建加工计划：AI收集信息 → 工具建草稿 → 既有微前端组件展示 → 用户确认 → 组件携认证信息直接调Java → 后端鉴权/业务校验、创建 → 组件展示结果并反馈Agent。

SSE断开≠业务失败；恢复后查原任务/草稿，不盲目重建。身份来自可信认证，不来自模型填写的userId；凭证不进入提示词。MCP统一发现/调用协议，不自动把所有REST接口变工具，也不能代替业务鉴权。

AI辅助研发：上下文收集 → OpenSpec/可读需求文档 → 人工审核和多方串讲 → 任务拆分/TDD → 测试与规范审核 → QC/测试/发布。自制分析和规范Skill并推广全组；不编造提效百分比。

9. 反问与临场收尾

选两问，不一次问完：

这个岗位主要负责哪块业务？前三个月希望我重点承担哪些工作？

团队主要是长期维护迭代，还是阶段性交付？当前阶段结束后，人员通常怎样安排？

HR阶段再确认固定月薪、额外薪资的发放条件、加班/值班安排和项目调整机制；不预设银行项目一定稳定或轻松。

进场前：确认地点/链接、设备和简历；提前留出时间。遇到不会的细节，先说确定的边界与排查方式，不把猜测讲成事实。